Лабораторная работа №5 "Разработка интерактивного интерфейса на JavaScript"

Лабораторная работа №5. Разработка интерактивного интерфейса на JavaScript

1. ЦЕЛЬ РАБОТЫ

Освоение основ языка JavaScript и практических навыков работы с DOM (Document Object Model). Формирование умения создавать интерактивные элементы веб-интерфейса, обрабатывать события и динамически изменять содержимое страницы **на примере проекта, тематически связанного с будущим курсовым проектом во 2 части курса (следующий семестр)**.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

Основы языка программирования JavaScript:

- Переменные, операторы, условные выражения и циклы.
- Массивы и объекты.
- Функции и обработка ошибок.

Работа с DOM:

- Основные методы для выборки элементов на странице.
- Изменение атрибутов, стилей и содержания элементов.
- Добавление и удаление элементов.

Обработка событий:

- Применение обработчиков событий (клики, отправка форм).

- Использование таймеров, анимаций и задержек.

Валидация форм и работа с LocalStorage.

Основы HTML и CSS (из предыдущих лабораторных работ).

Основы работы с консолью разработчика в браузере.

---

3. ЗАДАЧИ РАБОТЫ

- Изучить основные операции работы с DOM (поиск элементов, манипуляция контентом и стилями).
- Освоить обработку событий (`click`, `input`, `submit` и др.).
- Реализовать интерактивные компоненты **в соответствии с тематикой вашего курсового проекта**.
- Научиться работать с формами и валидацией данных.
- Организовать код в соответствии с лучшими практиками (ES6+, модульность).
- Использовать `LocalStorage` для сохранения состояния (например, корзины, избранного, лайков и т.п.).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ (общий для всех вариантов)

Часть 1. Подготовка проекта и настройка окружения

1. **Перейдите в рабочую ветку репозитория:**

```
git checkout main
git pull origin main
git checkout -b feature/javascript-interactivity
```

2. **Создайте структуру файлов JavaScript:**
    - В папке `js` создайте:
        - `script.js` – основной файл скриптов.
        - `components/` – папка для отдельных компонентов (например, `cart.js`, `filters.js`, `slider.js`).

    - `utils/` – папка для вспомогательных функций (например, `storage.js`, `validation.js`).

3. **Подключите JavaScript к HTML в конце тега `<body>`:**

```
<script src="js/script.js"></script>
```

Часть 2. Основы работы с DOM (примеры для ознакомления)

- Поиск элементов: `getElementById()`, `querySelector()`, `querySelectorAll()`.
- Манипуляция контентом: `textContent`, `innerHTML`, `createElement()`, `appendChild()`, `remove()`.
- Работа с классами: `classList.add()`, `classList.remove()`, `classList.toggle()`.

Часть 3. Обработка событий

- Базовые события: `click`, `input`, `submit`, `change`, `scroll`.
- Делегирование событий: использование `event.target` для обработки динамически добавляемых элементов.

Часть 4. Реализация интерактивных компонентов (по вашему проекту)

Общее задание для всех студентов (индивидуально):

На основе HTML-структуры и CSS-стилей, созданных в предыдущих лабораторных работах, реализуйте **не менее 3 различных типов событий** и **не менее 2 интерактивных компонентов**, характерных для вашей предметной области.

Примеры интерактивных компонентов (выберите подходящие для вашего проекта):

- Фильтрация элементов (новостей, товаров, рецептов, курсов и т.п.) по категориям, цене, тегам.
- Корзина покупок / избранное (добавление, удаление, подсчёт итогов) с сохранением в `LocalStorage`.
- Слайдер изображений (ручной или автоматический).
- Табы (вкладки) для переключения между разными типами контента.

- Модальное окно (для подробного просмотра, добавления, редактирования).
- Система лайков / оценок (звёздный рейтинг) с сохранением состояния.
- Форма с валидацией (регистрация, бронирование, отзыв, обратная связь).
- Аккордеон (раскрывающиеся блоки) для FAQ или категорий.
- Прогресс-бар (чтения статьи, прохождения курса, обратного отсчёта).
- Таймер (обратный отсчёт до мероприятия, время приготовления блюда).
- Бесконечная прокрутка или пагинация (динамическая подгрузка элементов).
- Поиск с автодополнением (по элементам на странице).

Требования:

- Каждый компонент должен использовать минимум один тип события (например, `click` для кнопки, `input` для поиска, `submit` для формы).
- Состояние интерактивных компонентов (например, содержимое корзины, избранное, лайки) должно сохраняться в `LocalStorage` и восстанавливаться при перезагрузке страницы.

Часть 5. Работа с формами и валидация (если применимо к вашему проекту)

- Реализуйте валидацию полей формы:
  - Проверка на пустоту.
  - Проверка формата (email, телефон, дата).
  - Вывод сообщений об ошибках.
- Отправка формы (имитация) – вывод данных в консоль или сохранение в `LocalStorage`.

Часть 6. Рефакторинг и тестирование

- Протестируйте функциональность в разных браузерах (Chrome, Firefox, Edge).
- Проверьте обработку ошибок (try/catch).
- Убедитесь, что код соответствует стандартам ES6+ (стрелочные функции, `const`/`let`, шаблонные строки).

Часть 7. Фиксация изменений

```
git add js/
git add index.html
git commit -m "feat: add JavaScript interactivity for [тип проекта]"
git push origin feature/javascript-interactivity
```

---

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист (по Стандарту Предприятия).
- Цель работы.
- Краткие теоретические сведения о DOM, событиях и основах JavaScript.
- **Ход работы:**
    - Описание выполненных действий (по пунктам 4.1–4.7).
    - Скриншоты:
        - Исходный код JavaScript в VS Code (основные функции).
        - Консоль браузера с выводами `console.log` (или без ошибок).
        - Работающее интерактивное поведение на странице (демонстрация каждого компонента).
        - Инструменты разработчика, показывающие обработчики событий (вкладка "Event Listeners").
    - Ссылка на коммит с выполненной работой.
- **Ответы на контрольные вопросы** (все вопросы).
- **Выводы** по работе.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.

- **Индивидуальное выполнение.** Все выполняют единый порядок действий (задание одинаково для всех вариантов, отличается только тематика проекта, которая выбирается в соответствии с будущим курсовым проектом).

  !!!! Для выбора темы используется Генератор вариантов

- Код должен быть загружен в репозиторий GitHub в ветке `feature/javascript-interactivity`.

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какие методы поиска элементов в DOM вы использовали в работе и в каких случаях каждый из них наиболее предпочтителен?
2. Как работает делегирование событий и какие преимущества оно даёт при работе с динамически добавляемыми элементами?
3. Какие способы создания и добавления новых элементов в DOM вы применяли? В чём разница между `innerHTML` и `createElement`?
4. Как вы организовали структуру JavaScript-кода в проекте? Какие функции вы вынесли в отдельные модули и почему?
5. С какими особенностями обработки событий вы столкнулись и как решили проблемы всплытия и перехвата событий?
6. Что такое DOM и как он связан с веб-страницей?
7. Как можно выбрать элемент по его идентификатору в DOM?
8. Опишите, как можно изменить текст внутри элемента с помощью JavaScript.
9. Что такое обработчик событий в JavaScript и как его добавить к элементу? Приведите пример.
10. Как получить доступ к значениям атрибутов элемента через JavaScript?
11. Какие существуют способы добавления нового элемента в DOM? Приведите пример кода.
12. Как удалить элемент из DOM?
13. Что такое `LocalStorage` и как с ним работать в JavaScript? Напишите пример сохранения и извлечения данных.

14. Как можно сохранять и извлекать данные из `LocalStorage` для вашего проекта (корзина, избранное, лайки)?
15. Как можно обновить содержимое страницы без её перезагрузки?
16. Какие методы JavaScript позволяют манипулировать стилями элементов на странице?
17. В чём заключается различие между методами `getElementById` и `querySelector`?
18. Как реализовать функциональность добавления нового элемента в список с помощью JavaScript?
19. Какие способы сортировки элементов на странице можно реализовать с помощью JavaScript?
20. Как реализовать простую форму валидации на JavaScript? Приведите пример проверки email.
21. Как можно изменить класс элемента с помощью JavaScript?
22. Как обрабатывать событие клика на кнопке и выполнить определённое действие?
23. Что такое событие `submit` и как с ним работать в JavaScript?
24. Как получить значения, введённые пользователем в текстовое поле формы?
25. Как реализовать фильтрацию элементов на странице с помощью JavaScript?
26. Что такое анимация в контексте JavaScript и как она может быть реализована (простой пример, например, плавное появление)?
27. Как реализовать слайдер изображений с помощью JavaScript (общая идея)?
28. Как создать выпадающий список с использованием JavaScript?
29. Как создать таймер, который отсчитывает определённое время в JavaScript?
30. Как реализовать функциональность перетаскивания элементов на веб-странице (drag & drop)?

---

8. ПРАКТИЧЕСКИЕ РЕКОМЕНДАЦИИ

- Используйте `console.log()` для отладки кода.
- Обрабатывайте возможные ошибки с помощью `try-catch`.

- Используйте современный синтаксис ES6+ (стрелочные функции, деструктуризацию, шаблонные строки).
- Следите за производительностью при работе с большим количеством элементов.
- Тестируйте код в разных браузерах (Chrome, Firefox, Edge).
- Для сохранения состояния в `LocalStorage` используйте `JSON.stringify()` и `JSON.parse()`.

---

9. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Созданная в этой работе интерактивность будет расширена в следующих лабораторных работах (№6 – асинхронные операции и API, №7 – взаимодействие с сервером и т.д.).
- Если выбранные вами компоненты не требуют `LocalStorage` (например, просто слайдер), всё равно добавьте хотя бы одно сохраняемое состояние (например, запоминать последний открытый слайд или активную вкладку).